Real-Time Stock Market Watchlist & Alert System

Review 3 – Presenter's Guide

BCSE302P Database Systems Lab · Dr. Anand Bihari
Aditi Bajpai (24BCE0372) · Arshmeet Kaur Marhala (24BCE0378) · Yatharth Tongia (24BCE0825)

For the team only – not part of the submitted report.
Read sections 1–3 first. Section 16 is the live demo script. Section 18 answers the viva questions.

Contents

1. The project in one minute
2. Words you must know
3. How the system works (architecture and the life of one price)
4. The ER diagram and what we fixed
5. The 7 tables
6. Normalization (1NF → BCNF) with proofs
7. Constraints and the ON DELETE policy
8. The alert engine (trigger) – worked example from the replay
9. The SQL queries in the report
10. Views, procedures, functions, triggers and cursors
11. Security: roles, GRANT and REVOKE
12. Transactions and concurrency
13. Indexing and query optimization
14. Testing
15. The web application – module by module
16. Live demo script (10–12 minutes)
17. Walking through the report, section by section
18. Viva questions with answers
19. Honest limitations – what not to claim
20. What changed in Phases 11, 12 and 13
21. Where everything is (file map)

1. The project in one minute

Share prices change every second. A casual investor cannot watch the screen all day. Our system lets a user keep **watchlists** (named lists of stocks or crypto pairs) and set **alert rules** such as 'tell me when RELIANCE goes ABOVE 3000'. Every new price is stored in PostgreSQL. A **trigger** inside the database compares the new price with the previous one and, when the price crosses a rule's threshold, it records an **alert**. The web page shows the watchlists, prices and alerts.

The key idea to say in the viva: **all the data rules live in the database**. Keys, CHECK constraints, the function that stores prices (`ingest_tick()`), the alert trigger and the index are all in PostgreSQL. Python and JavaScript only send requests and show results. So the rules are the same no matter who sends the data – the website, the replay script, the live Binance feed or psql.

What it is NOT: not a trading app (it never buys or sells), not a portfolio tracker, not a price predictor. It is a watchlist + alert system. Say this if anyone asks about orders or advice.

Numbers to remember

What	Number
Tables	7 (all in BCNF)
Named constraints	36 = 7 PRIMARY KEY + 8 FOREIGN KEY + 7 UNIQUE + 14 CHECK
Seed data	3 users, 7 instruments, 5 watchlists, 11 items, 6 rules (5 active), 0 ticks, 0 alerts
Clean-state check line	3|7|5|11|6|0|0
Replay run	30 ticks inserted, 0 duplicates, 0 errors, 8 alerts
SQL queries in report	38 (A1–G3) + 10 complex (CQ01–CQ10)
Views / procedures / functions	3 views · 3 procedures · 3 reporting functions + ingest_tick() + 2 trigger functions
Triggers	2
Roles	sw_admin, sw_analyst, sw_enduser + logins arjun, priya
Index	ix_price_ticks_instrument_time (instrument_id, observed_at DESC, tick_id DESC)
Optimization	latest 50 ticks on 500,000 rows: 19.966 ms → 0.100 ms; 19 MB index
Tests	9 suites, 221 checks (51 + 50 + 17 + 18 + 15 + 32 + 14 + 24) + schema verifier – all pass
Web modules	6: Dashboard, Watchlists, Alert Rules, Alert History, Instruments, Demo

2. Words you must know

Word	Simple meaning	Example in our project
Instrument	Something whose price we follow: a stock or a crypto pair, on one exchange.	NSE RELIANCE, BINANCE BTCUSDT
Tick (price tick)	One price of one instrument at one moment.	RELIANCE 3002.40 at 07:38:07 PM
Watchlist	A named list of instruments that a user owns.	arjun's 'Crypto' list
Alert rule	'Tell me when price goes ABOVE / BELOW a threshold.' Has a cooldown.	Rule #1: RELIANCE ABOVE 3000, cooldown 300 s
Alert event	A record that a rule fired on a particular tick.	event 1: rule #1 fired on tick 6
Crossing (edge-	Fire only when the price moves from one side of	2994 → 3002.40 crosses 3000

triggered)	the threshold to the other – not on every tick that is above.	
Cooldown	Minimum time (in market time) between two alerts of the same rule.	300 s for rule #1
Late tick	A tick whose time is older than a tick we already have. It is stored but not checked.	test 13c
Replay feed	A Python script that reads 30 prices from a CSV file, so the demo works offline.	app/replay.py
ingest_tick()	The PostgreSQL function every feed uses to store a price. It locks the instrument, inserts the tick or reports DUPLICATE.	sql/03_functions_triggers.sql
Trigger	Code that PostgreSQL runs automatically when a row is inserted/updated.	trg_price_ticks_evaluate_alerts
Surrogate key	A made-up number used as the primary key.	user_id, tick_id
Natural / candidate key	A real-world column set that is also unique.	(exchange, symbol)
CASCADE / RESTRICT	What happens to child rows when a parent is deleted: delete them too / refuse the delete.	user → watchlists CASCADE; instrument → ticks RESTRICT
BCNF	Every column set that decides another column is a key.	all 7 tables
Index (B-tree)	A sorted structure that lets PostgreSQL find rows without reading the whole table.	ix_price_ticks_instrument_time
Transaction	A group of statements that all happen or none happen.	BEGIN … COMMIT / ROLLBACK
Row lock	A session marks a row as 'in use'; others who want to change it must wait.	FOR NO KEY UPDATE in ingest_tick()
Role / GRANT / REVOKE	A database user or group / give a right / take a right away.	sw_analyst can read prices only

3. How the system works

3.1 The parts

```
Browser (HTML + CSS + JavaScript)  -->  FastAPI (Python, app/main.py)  -->  PostgreSQL
18
                                                                            - 7 tables,
36 constraints
CSV replay feed  (app/replay.py)    --+                                     -
ingest_tick()
Binance live feed (app/live_feed.py)--+--> ingest_tick() ------------------> - alert
trigger -> alert_events
                                                                            - index on
price_ticks
```

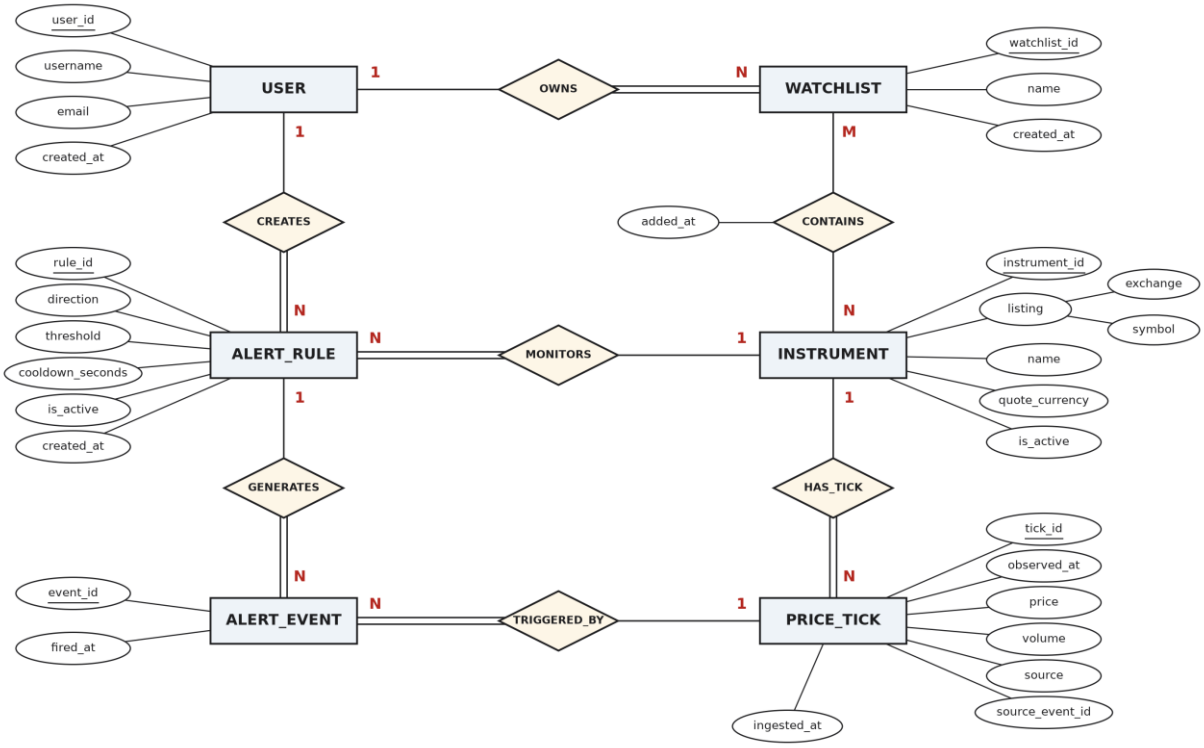
- **PostgreSQL 18** – the database. All rules and the alert logic are here.
- **FastAPI** (Python) – a small web server with 18 endpoints, e.g. `GET /users/{id}/watchlists`, `POST /ticks/ingest`. It uses psycopg 3 with parameterized SQL (no ORM).
- **Web page** – plain HTML, CSS and JavaScript (no framework). It calls the API.
- **Replay feed** – reads `data/replay_prices.csv` (30 rows) and calls `ingest_tick()` for each row.

- **Live feed (optional)** – reads real crypto trades from Binance's public WebSocket. Not needed for the demo.

3.2 The life of one price (say this in the demo)

22. A price arrives (from the replay, the live feed, or the Demo page): e.g. RELIANCE 3002.40.
23. The program calls `ingest_tick('NSE','RELIANCE', time, 3002.40, volume, 'REPLAY', event_id)`.
24. `ingest_tick()` finds RELIANCE and **locks its row** (`SELECT … FOR NO KEY UPDATE`). Another price for RELIANCE must now wait; other instruments are not blocked.
25. It inserts the tick. If the same (source, instrument, event id) is already stored, nothing is inserted and it returns **DUPLICATE**.
26. The **AFTER INSERT trigger** runs. It finds the previous RELIANCE tick (using the index), and checks each active rule on RELIANCE: did the price cross the threshold? Is the cooldown over?
27. If yes, it inserts a row into `alert_events` (rule_id, tick_id, fired_at) and sends `pg_notify` with the event_id.
28. The transaction commits: the tick and its alert are saved together. The web page shows it on the next refresh.

4. The ER diagram and what we fixed



The corrected ER diagram (report Figure 5.1)

4.1 How to read it

- **Rectangle** = entity type (a 'thing'): USER, WATCHLIST, INSTRUMENT, PRICE_TICK, ALERT_RULE, ALERT_EVENT.
- **Diamond** = relationship: OWNS, CONTAINS, CREATES, MONITORS, HAS_TICK, GENERATES, TRIGGERED_BY.

- **Oval** = attribute. **Underlined** = key attribute. `listing` is a composite attribute made of exchange + symbol.
- **1 / N / M** next to a line = cardinality (the maximum). USER 1 — N WATCHLIST means one user, many watchlists.
- **Double line** = total participation (the minimum is 1): every WATCHLIST must have an owner, because `watchlists.user_id` is NOT NULL. A single line = partial: a user may have no watchlist.

4.2 The three fixes asked for after Review 2

Feedback	What was wrong	What we did and how to defend it
No arrows	The old draw.io diagram had arrow heads on every line.	All lines are plain. Chen notation has no arrows; cardinality is written as 1/N/M and participation as single/double lines.
Wrong weak entity	Watchlist_item was a double rectangle (weak entity).	A weak entity needs its own **partial key** and an **identifying relationship** to an owner. Watchlist_item has no partial key – it is identified only by the watchlist and instrument it links, and it has one attribute (added_at). That is exactly an **M:N relationship with an attribute**. So it is now the diamond CONTAINS between WATCHLIST and INSTRUMENT.
Remove join-only attributes	watchlist_id and instrument_id were drawn as attributes of Watchlist_item.	Foreign keys are not attributes in an ER model. A link is shown by a relationship diamond. The FK columns appear only when we **map** the ER diagram to tables (section 6 of the report). So WATCHLIST has no user_id oval, PRICE_TICK has no instrument_id oval, ALERT_EVENT has no rule_id/tick_id.

Likely follow-up: "Then why do entities still have user_id, tick_id, etc.?" – Those are each entity's **own key** (its identifier), not join columns. A strong entity must have a key attribute. What we removed are the **foreign keys** – columns that only point to another entity.

4.3 Other questions about the diagram

Q. Why is ALERT_EVENT an entity and not a relationship between ALERT_RULE and PRICE_TICK?

Conceptually an alert is 'rule R fired on tick T', so it could be an M:N relationship. We keep it as an (associative) entity because it has its own identity used on its own: `event_id` is sent by `pg_notify` and the Alert History lists alerts by it. Its uniqueness rule (one alert per rule per tick) is UNIQUE (rule_id, tick_id) in the table.

Q. Is WATCHLIST a weak entity, since its name is unique only per user?

It could be modelled that way (partial key = name). But it has its own key `watchlist_id`, which other tables use, so it is a strong entity. The per-user rule is UNIQUE (user_id, name).

Q. How does the ER diagram become tables?

- Each entity → a table with its attributes; the key → PRIMARY KEY.
- Each 1:N relationship → a FOREIGN KEY column on the N side (e.g. OWNS → watchlists.user_id NOT NULL because participation is total).
- The M:N relationship CONTAINS → a new table watchlist_items(watchlist_id, instrument_id, added_at) with PK (watchlist_id, instrument_id).
- The composite attribute listing → two columns exchange, symbol with UNIQUE (exchange, symbol).

Result: 6 entities + 1 M:N relationship = 7 tables.

5. The 7 tables

Table	One row is…	Keys	Important rules
users	a demo user	PK user_id; UNIQUE username; UNIQUE email	username 3–30 lower-case chars (CHECK); email lower-case
instruments	one listing (symbol on an exchange)	PK instrument_id; UNIQUE (exchange, symbol)	exchange IN (NSE, BSE, BINANCE); is_active; no asset_class column
watchlists	a named list of one user	PK watchlist_id; UNIQUE (user_id, name)	FK user_id → users ON DELETE CASCADE
watchlist_items	one instrument in one list	PK (watchlist_id, instrument_id)	FK to watchlists CASCADE; FK to instruments RESTRICT; added_at
price_ticks	one price at one moment	PK tick_id; UNIQUE (source, instrument_id, source_event_id)	price NUMERIC > 0; volume may be NULL (unknown) but ≥ 0; source IN (REPLAY, BINANCE, MANUAL); append-only; FK instrument RESTRICT
alert_rules	one user's alert rule	PK rule_id; UNIQUE (user_id, instrument_id, direction, threshold)	direction ABOVE/BELOW; threshold > 0; cooldown ≥ 0; is_active; only is_active and cooldown can be changed
alert_events	one alert that fired	PK event_id; UNIQUE (rule_id, tick_id)	only 4 columns – never copies price, time or instrument; trigger checks rule and tick are the same instrument

Design choices to defend

- **NUMERIC, not FLOAT, for prices** – exact decimals. 0.1 + 0.2 = 0.3 exactly; with FLOAT it is 0.30000000000000004, so 'price ≥ 3000' could be wrong at the boundary.
- **TIMESTAMPTZ** – stores an exact moment; NSE (IST) and Binance (UTC) times compare correctly.
- **observed_at vs ingested_at** – market time of the trade vs the time we stored it. Alerts use market time.
- **volume NULL means unknown**, not zero. Zero would mean 'no trading happened' and would change averages.
- **No latest_price table** – the latest price is the newest tick; the index makes this fast. A copy would need to be kept in sync (update anomaly).
- **No UNIQUE (instrument_id, observed_at)** – two real trades can happen at the same time at different prices.
- **Rules are not edited**, only disabled and recreated – so old alerts still match the rule definition that fired them.

6. Normalization (1NF → BCNF)

6.1 The normal forms in one line each

Form	Rule	Our example
1NF	One value per cell, no lists.	Instruments of a watchlist are separate rows in watchlist_items, not 'RELIANCE,TCS' in one cell.
2NF	No non-key column depends on only PART of a key.	watchlist_items does not store user_id (it depends only on watchlist_id).
3NF	No non-key column depends on another non-key column.	alert_events does not store price. From the PK it would be transitive (event_id → tick_id → price); and because tick_id is part of the key (rule_id, tick_id) it would already break 2NF.
BCNF	Every determinant (left side of an FD) is a candidate key.	True for all 7 tables.

6.2 Functional dependencies and keys

Table	Candidate keys	Main FDs
users	{user_id}, {username}, {email}	each key → all other columns
instruments	{instrument_id}, {exchange, symbol}	(exchange, symbol) → name, quote_currency, is_active
watchlists	{watchlist_id}, {user_id, name}	(user_id, name) → watchlist_id, created_at
watchlist_items	{watchlist_id, instrument_id}	(watchlist_id, instrument_id) → added_at
price_ticks	{tick_id}, {source, instrument_id, source_event_id}	tick_id → observed_at, price, volume, ingested_at, …
alert_rules	{rule_id}, {user_id, instrument_id, direction, threshold}	rule_id → cooldown_seconds, is_active, created_at, …
alert_events	{event_id}, {rule_id, tick_id}	event_id → rule_id, tick_id, fired_at

6.3 Attribute closure example (the viva may ask you to do this on the board)

Take instruments(instrument_id, exchange, symbol, name, quote_currency, is_active) with FDs F1: instrument_id → exchange, symbol, name, quote_currency, is_active and F2: exchange, symbol → instrument_id.

29. Start: {exchange, symbol}$^+$ = {exchange, symbol}.
30. Apply F2: add instrument_id → {exchange, symbol, instrument_id}.
31. Apply F1: add name, quote_currency, is_active → all 6 columns.
32. So {exchange, symbol} is a superkey. Neither exchange alone nor symbol alone gives all columns (5 NSE rows share 'NSE'; BSE RELIANCE is legal), so it is a minimal key – a candidate key.

6.4 FDs we deliberately REJECTED

- **exchange → quote_currency** is false: Binance quotes BTCUSDT in USDT but ETHBTC in BTC. (Screenshot N01 adds ETHBTC in a rolled-back transaction. That shows the database does not *enforce* the FD; the reason we reject it is the business fact, not the insert.)
- **symbol → name** is *not adopted*: symbols are unique only within an exchange, so the design does not assume one symbol means one company everywhere. Do **not** use BSE RELIANCE as a

counterexample – it is the same company with the same name, so it agrees with the FD (this was corrected in Phase 13). N01's BSE RELIANCE row only shows that the same symbol may be listed on two exchanges.

- **exchange → asset_class**: that is why there is no asset_class column. If it existed and depended on exchange, it would depend on part of the key (exchange, symbol) – a 2NF problem. It is also false (NSE lists ETFs too).
- **(instrument_id, observed_at) → price** is false: two trades can share a time.

6.5 Anomaly if two tables were merged

Suppose we merged alert_events with price_ticks and stored price, instrument and time inside every alert row. If a tick's price had to be corrected, we would have to change it in price_ticks **and** in every alert that fired on it – miss one and the data disagrees (**update anomaly**). Deleting the last alert would not lose the tick, but we could not store an alert without copying the tick (**insertion** problem). Keeping alert_events to (event_id, rule_id, tick_id, fired_at) avoids this; we get price and time by joining.

7. Constraints and the ON DELETE policy

36 named constraints: 7 PRIMARY KEY, 8 FOREIGN KEY, 7 UNIQUE, 14 CHECK. Every one has a name (e.g. `uq_watchlists_user_name`) so an error says exactly which rule was broken, and the API can turn it into a readable message (HTTP 409 'This user already has a watchlist with that name').

Parent → child	ON DELETE	Why
users → watchlists	CASCADE	a user's own data goes with the user
users → alert_rules	CASCADE	same
watchlists → watchlist_items	CASCADE	deleting a list deletes its entries (never the instruments or prices)
alert_rules → alert_events	CASCADE	a rule's alert history belongs to the rule
instruments → price_ticks	RESTRICT	market history is shared; must not disappear by accident
instruments → watchlist_items	RESTRICT	cannot delete an instrument somebody watches
instruments → alert_rules	RESTRICT	cannot delete an instrument a rule monitors
price_ticks → alert_events	RESTRICT	an alert must always be able to show its tick

"What disaster if swapped?" – If instruments → price_ticks were CASCADE, deleting one instrument by mistake would silently delete its whole price history and (through price_ticks → alert_events) everybody's alerts. If users → watchlists were RESTRICT, you could never delete a user without first deleting all their lists by hand.

CHECK and NULL: `CHECK (volume >= 0)` lets NULL through, because NULL >= 0 is 'unknown', and CHECK only rejects FALSE. That is what we want for volume (unknown is allowed). For price we add NOT NULL, so `price NUMERIC NOT NULL CHECK (price > 0)` rejects both NULL and ≤ 0.

8. The alert engine – worked example from the replay

Rule logic: ABOVE fires when **previous < T and new ≥ T**; BELOW fires when **previous > T and new ≤ T**. The first tick of an instrument never fires (no previous price). Cooldown is measured in market time (observed_at). A late tick is stored but not evaluated. Disabled rules are ignored.

RELIANCE in the replay (rule #1 ABOVE 3000, cooldown 300 s; rule #2 BELOW 2800)

Time	Price	What happens
0 s	2985.50	first tick – no previous price, no alert
10 s	2994.00	still below 3000
20 s	3002.40	**2994 → 3002.40 crosses 3000 → event 1 (rule #1)**
30 s, 45 s	3011.00, 3018.75	stays above – no new alert (edge-triggered)
90 s	2979.00	drops below 3000 – ABOVE rule does not fire on a fall
200 s	3004.00	crosses 3000 again, but only 180 s since the last alert (< 300 s cooldown) → **suppressed**
240 s	2990.00	below
400 s	3021.50	**crosses again, 380 s since last alert → event 7**
430 s	3030.00	stays above
450 s	2795.00	**3030 → 2795 crosses BELOW 2800 → event 8 (rule #2)**
460 s	2788.00	stays below – no new alert

The other 5 alerts

- Event 2: priya TCS BELOW 3500 – 3531.25 → 3498 at 35 s.
- Event 3: arjun BTCUSDT ABOVE 100000 – 99880 → 100120 at 40 s. (At 80 s BTC crosses again 99920 → 100050, but only 40 s after the last alert and the cooldown is 60 s → suppressed.)
- Event 4: kavya ETHUSDT BELOW 3000 – 3048.10 → 2994.60 at 110 s.
- Event 5: priya TCS BELOW 3500 – 3512 → 3490 at 130 s (cooldown is 0, so it can fire again).
- Event 6: arjun BTCUSDT ABOVE 100000 – 99800 → 100400 at 170 s (130 s after event 3 – cooldown over).
- INFY goes 1585 → 1612 and crosses 1600, but rule #5 is **disabled** → no alert. Total: **8 alerts**.

9. The SQL queries in the report

Every query is a file in `sql/screenshots/` and every screenshot shows the command, the SQL and the real output. Know one sentence for each category:

Category	What to say
A. Basic (A1–A6)	WHERE filters rows, ORDER BY sorts, DISTINCT removes repeats, BETWEEN includes both ends, IN is a short OR, LIKE/ILIKE match patterns, IS NULL finds unknown values (= NULL never works).
B. Aggregate (B1–B5)	GROUP BY makes groups; COUNT/SUM/AVG/MIN/MAX per group. **B4: WHERE filters rows before grouping, HAVING filters groups after.** COUNT(*) counts rows, COUNT(volume) skips NULLs.
C. Joins (C1–C6)	INNER keeps matches only; LEFT keeps all left rows (instruments with 0 ticks show 0); RIGHT keeps all right rows; FULL keeps both; SELF joins a table to itself (two users watching the same instrument); CROSS makes every pair (3 users × 7 instruments = 21).
D. Subqueries (D1–D8)	Scalar returns one value; single-row compared with =; multi-row with IN/ANY/ALL; correlated runs per outer row; EXISTS / NOT EXISTS test for any row. **D8: NOT IN with a NULL in the list returns no rows – use NOT EXISTS.**
E. Set operations (E1–	UNION removes duplicates, UNION ALL keeps them (faster, no sort), INTERSECT = in

E4)	both, EXCEPT = in first but not second (PostgreSQL's MINUS).
F. Functions (F1–F6)	CASE = if/else; COALESCE = first non-NULL; NULLIF(x,0) avoids divide-by-zero; date/time, string and numeric functions.
G. Window (G1–G3)	ROW_NUMBER/RANK/DENSE_RANK (differ on ties); running total with SUM() OVER (ORDER BY …); LAG/LEAD give previous/next row – LAG is exactly the comparison our trigger makes.
CQ01–CQ10	Business questions over 3–5 tables, e.g. CQ09 proves each of the 8 alerts was a genuine crossing; CQ06 shows which DISABLED rules would have fired.

10. Views, procedures, functions, triggers and cursors

All Review 2 objects are in a separate schema called `reporting` (file `sql/09_review2_objects.sql`), so the 7 core tables were not changed.

Object	Name	What it does / why
View V1	reporting.v_watchlist_contents	joins 4 tables: who watches what – simplifies a common query
View V2	reporting.v_instrument_price_summary	aggregate per instrument (count, min, max, avg) – for the analyst role
View V3	reporting.v_my_alerts	security view: WHERE username = current_user, WITH (security_barrier) – each end user sees only their alerts
Procedure	sp_create_watchlist(user, name, exchange, symbols[])	creates a list and its items **all or nothing**: one bad symbol → exception → nothing created
Procedure	sp_set_rule_active(rule_id, on/off)	enables/disables a rule; error if the rule does not exist
Procedure	sp_print_watchlist_report(user)	REFCURSOR: OPEN … FOR, FETCH loop, RAISE NOTICE each line, CLOSE
Function	fn_latest_price(instrument_id)	latest price or NULL; uses the index
Function	fn_price_change_pct(instrument_id)	% change first → latest tick; calls fn_latest_price
Function	fn_rule_status_report(user)	explicit parameterized cursor: DECLARE c_rules CURSOR (p_user), OPEN, FETCH, CLOSE
Function	ingest_tick(…)	the only way prices enter; lock + insert/DUPLICATE; errors SW001 unknown, SW002 inactive
Trigger	trg_price_ticks_evaluate_alerts	AFTER INSERT on price_ticks, FOR EACH ROW: crossing + cooldown → alert_events, pg_notify
Trigger	trg_alert_events_check_instrument	BEFORE INSERT/UPDATE on alert_events: rule and tick must be the same instrument

Updatable views? None of the 3 views is automatically updatable – they use joins or GROUP BY (screenshot V4 shows information_schema and the failed UPDATE).

Function vs procedure: a function returns a value and can be used inside SELECT; it cannot COMMIT. A procedure is run with CALL and may control transactions. fn_latest_price is a function so it can be used in a query like `SELECT symbol, reporting.fn_latest_price(instrument_id) FROM instruments`.

11. Security: roles, GRANT and REVOKE

Role	Can do	Cannot do
sw_admin (group)	SELECT/INSERT/UPDATE/DELETE on all tables, all reporting objects	–
sw_analyst (group)	read instruments, price_ticks, v_instrument_price_summary; run fn_latest_price, fn_price_change_pct	read users or rules; change anything
sw_enduser (group)	read reporting.v_my_alerts only	read any base table
arjun, priya (logins)	members of sw_enduser – each sees only their own alerts	see other users' alerts; call sp_set_rule_active

- **Least privilege** (SEC5): `SET ROLE sw_analyst; DELETE FROM price_ticks;` → permission denied.
- **REVOKE** (SEC4): before – analyst reads price_ticks; after `REVOKE SELECT` – permission denied; ROLLBACK restores it.
- **Secure exposure**: end users have no table rights at all; the view v_my_alerts is the only way in.
- `REVOKE EXECUTE ON ALL ROUTINES … FROM PUBLIC` – by default anyone may run a function; we close that first.

Be precise: `SET ROLE` switches the rights in force inside an already-logged-in session. It is not a new login. Our roles have no passwords; the demo uses SET ROLE from the postgres session.

12. Transactions and concurrency

12.1 ACID with our own examples

Property	Meaning	In our project
Atomicity	all or nothing	a tick and the alert its trigger creates commit or roll back together; sp_create_watchlist creates all items or none
Consistency	constraints hold at commit	36 constraints + 2 triggers reject bad data
Isolation	sessions do not see each other's half-done work	READ COMMITTED + the row lock in ingest_tick()
Durability	committed data survives a crash	PostgreSQL's write-ahead log (WAL) is flushed (fsync) at COMMIT

12.2 The concurrency demo (R3_03) step by step

```
stock-watchlist> python scripts\review3_concurrency_demo.py stock_watchlist_review3

Database: stock_watchlist_review3   isolation level: read committed
Rule #1: arjun  RELIANCE ABOVE 3000

=== Scenario 1: A and B send RELIANCE prices at the same time ===
[ 0.04s] setup    baseline tick 2990 committed -> INSERTED tick 34
[ 0.08s] A        BEGIN; ingest_tick(3005) -> INSERTED tick 35, alerts fired: 1  (row lock held)
[ 1.07s] B        BEGIN; ingest_tick(3010) ... waiting for the RELIANCE lock
[ 2.08s] observer pg_stat_activity: cc_B wait_event_type=Lock, blocked by cc_A
[ 3.08s] A        COMMIT  (lock released)
[ 3.09s] B        returned after waiting 2.02s -> INSERTED tick 36, alerts fired: 0
[ 3.09s] B        COMMIT

Result: new alert events for rule #1 = 1 (exactly one alert for one real crossing; no duplicate alert)

=== Scenario 2: the SAME event is sent by two sessions at once ===
[ 3.24s] A        ingest_tick(source_event_id 'cc:dup') -> INSERTED tick 37
[ 3.24s] B        ingest_tick(source_event_id 'cc:dup') -> DUPLICATE

Result: rows stored for 'cc:dup' = 1 (UNIQUE (source, instrument_id, source_event_id))

stock-watchlist>
```

Screenshot R3_03

33. Setup: RELIANCE baseline price 2990 is committed. Rule #1 is ABOVE 3000.
34. Session A: BEGIN; sends 3005. 2990 → 3005 crosses 3000 → 1 alert. A **keeps the RELIANCE row locked** for 3 seconds (does not commit yet).
35. Session B (1 s later): BEGIN; sends 3010. `ingest_tick()` tries to lock RELIANCE → **waits**.
36. Observer (2 s): pg_stat_activity shows cc_B wait_event_type = Lock, blocked by cc_A.
37. A commits (3 s). The lock is released. B continues after waiting 2.02 s.
38. B now sees 3005 as the previous price. 3005 → 3010 is not a crossing → **0 alerts**. Total: exactly 1 alert for 1 real crossing.
39. Without the lock, B would read the last *committed* price 2990 (READ COMMITTED does not show A's uncommitted 3005), see 2990 → 3010 as a crossing, and create a **duplicate alert**. That is the race condition we prevent.
40. Scenario 2: both sessions send the same event id at once – one INSERTED, one DUPLICATE (UNIQUE constraint). Only 1 row stored.

Command to run it again (needs the replayed demo database):

```
python scripts/review3_concurrency_demo.py stock_watchlist
```

Run it **once, as the last demo step**, on a freshly reset + replayed database. Its ticks are stamped 2 hours ahead and use fixed event ids, so a second run gives DUPLICATE results, and for 2 hours afterwards manual RELIANCE ticks count as late. Reset the database after the demo.

12.3 Isolation anomalies – know the table

Anomaly	Meaning	Prevented at
Dirty read	reading another session's uncommitted change	READ COMMITTED and above (PostgreSQL never allows it)
Non-repeatable read	the same row read twice gives different values	REPEATABLE READ and above
Phantom read	the same query returns new rows the second time	SERIALIZABLE (PostgreSQL's REPEATABLE READ also blocks it)

Lost update	two sessions read, both write, one change is lost	row locks (SELECT … FOR UPDATE), REPEATABLE READ retry, or a version column

At our default level (READ COMMITTED) non-repeatable reads and phantoms are possible, which is why ingest_tick() takes an explicit row lock for the one place where it matters.

13. Indexing and query optimization

- Index: `ix_price_ticks_instrument_time ON price_ticks (instrument_id, observed_at DESC, tick_id DESC)`.
- Used by: latest price, price history, the watchlist page, fn_latest_price, and the trigger's 'previous tick' lookup.
- **Column order:** equality column first (instrument_id), then the sort columns. So the newest ticks of one instrument sit together at the front of the index – PostgreSQL reads 1 or 50 entries and stops. No Sort step.
- **WHERE observed_at = ? alone will NOT use it well** – the first column is instrument_id. A B-tree is like a phone book sorted by surname then first name: you cannot look up by first name alone.
- **Numbers (IX2 → IX3, 500,000 rows):** Bitmap Heap Scan + Sort reading ~25,000 rows, 19.966 ms → Index Scan reading 50 rows, 0.100 ms. Buffers ~6,371 → 16 (benchmark medians).
- **Cost:** 19 MB on disk (table 48 MB) and one extra index write per INSERT. Five more indexes would make every tick insert slower and use more space – only add an index that a real query needs.
- **Why test on 500,000 rows?** On 30 rows PostgreSQL may just read the whole table (a sequential scan is cheaper than an index for tiny tables). The benefit only shows on realistic sizes.
- **BRIN** was compared in the benchmark and rejected: it is small but not per instrument and not ordered, so it does not help 'latest N of one instrument'.

Reading EXPLAIN ANALYZE

- Each line is a step (node); indented lines feed the line above.
- `Index Scan using ix_price_ticks_instrument_time` = reading through our index.
- `Sort` = PostgreSQL had to sort – the index removes this step.
- `actual time=… rows=…` = what really happened; `cost=` = the planner's estimate.
- `Buffers: shared hit=16` = 16 pages read from memory.
- Joins: Nested Loop (small inner side or index), Hash Join (build a hash table of one side), Merge Join (both sides sorted).

14. Testing

Suite	Checks	What it proves
sql/02_schema_tests.sql	51	every UNIQUE, CHECK, FK, RESTRICT and CASCADE rule (rolled back)
sql/verify_spec.sql	–	the live schema matches the design: columns, 36 constraints, objects
sql/04_alert_tests.sql	50	crossing, equality, cooldown, duplicates, late ticks, same-time ticks, savepoints
tests/concurrency_test.sh	17	real parallel psql sessions: lock waiting, no duplicate alerts, NOTIFY

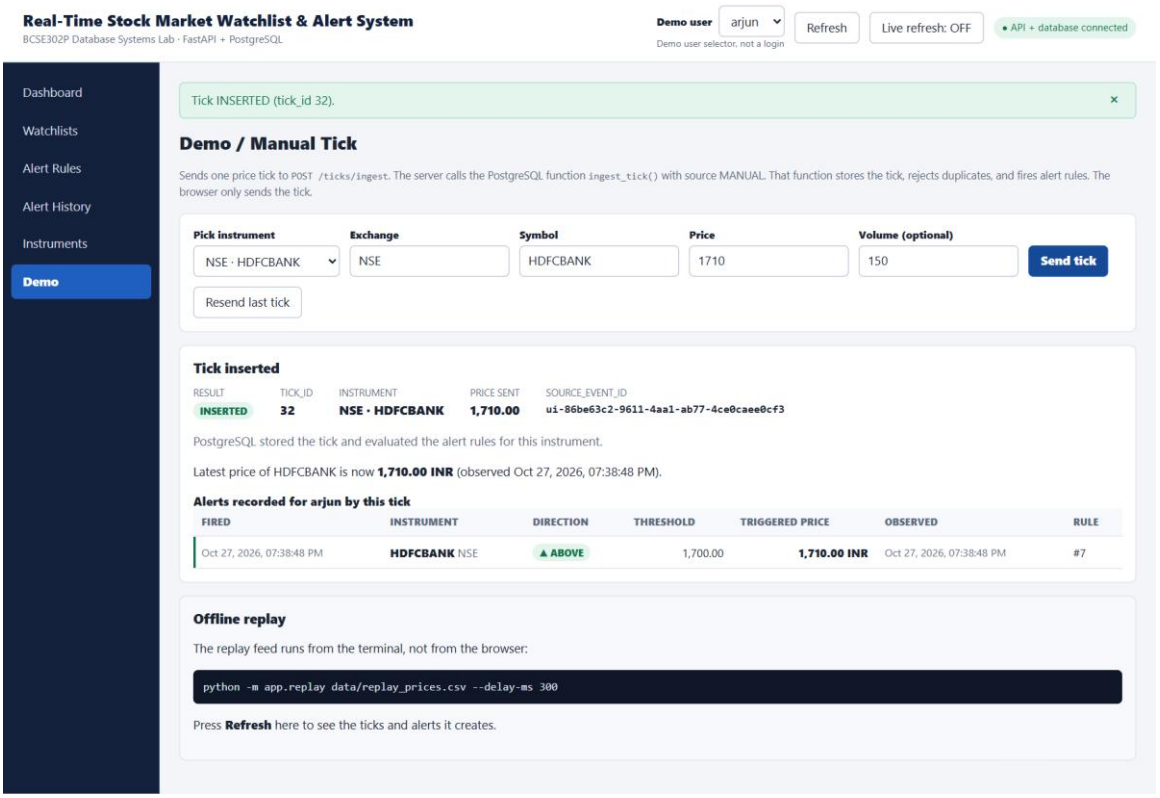
tests.test_replay	18	replay inserts 30 / 8 alerts, idempotent rerun, errors
tests.test_indexes	15	the index exists with the right columns and is used by the plans
tests.test_api	32	all endpoints, error messages, SQL injection is stored as text
tests.test_frontend	14	the web files are served and wired up
tests.test_live_feed	24	Binance message parsing (no internet needed)

Run all: `bash scripts/run_all_tests.sh` → ALL SUITES PASSED (screenshot R3_04). SQL suites roll back and Python suites use temporary databases, so the demo database is not changed.

Windows note (be honest if asked): on this Windows laptop two SQL test files use `\o /dev/null`, which Windows psql cannot open, and Git checks files out with CRLF line endings. With `\o NUL` and LF line endings every suite passes – that is how R3_04 was produced. These are test-file portability issues, not bugs in the application.

15. The web application – module by module

Module	Input (what you do)	Output (what you see)	Screenshot
Dashboard	pick the demo user	counts, 5 latest alerts, watchlists with latest prices	UI01, UI10
Watchlists	type a name → Create; choose instrument → Add; Remove; Delete watchlist	lists with latest price and time per instrument	UI02, UI03 (duplicate name error)
Alert Rules	instrument, ABOVE/BELOW, threshold, cooldown → Create; Disable/Enable; Save cooldown	rules table with ACTIVE/DISABLED	UI04
Alert History	optional instrument filter	alerts from alert_events + rule + tick, newest first	UI08
Instruments	search box, exchange filter, 'Latest price & history'	latest price, chart, tick list	UI09
Demo	pick instrument, price, volume → Send tick; Resend last tick	INSERTED / DUPLICATE, tick_id, alerts fired by this tick	UI05, UI06, UI07
API docs	open /docs	Swagger page with all endpoints	UI11



UI06 – a manual tick that fires an alert

16. Live demo script (10–12 minutes)

Golden rule order: reset → start app → replay → manual ticks. Replay ticks are stamped up to ~7 min 40 s into the future. For about 8 minutes after a replay, a manual tick for RELIANCE, TCS, INFY, BTCUSDT or ETHUSDT is older than the newest tick, so it is treated as a **late tick** (stored, no alert). **Use HDFCBANK for the manual-tick demo** – it has no replay ticks. Also do not start the live Binance feed after a replay (it refuses on purpose).

Before the review (once)

```
# Git Bash, in the project folder
export PG_BIN="/c/Program Files/PostgreSQL/18/bin"
export PATH="$PG_BIN:$PATH"
export PGHOST=localhost PGPORT=5432 PGUSER=postgres
source .venv/Scripts/activate
```

Open: one Git Bash terminal for commands, one for the server, a psql window, and the browser. Rehearse once end-to-end and time it.

Step-by-step

Min	Step (template demo flow)	Do this	Say this
0–1	Intro	Show the ER diagram (report Fig 5.1).	6 entities, 7 relationships, 7 tables; what we fixed after Review 2.
1–3	(1) DDL + DML	`bash scripts/reset_demo_db.sh` → ends with OK …	Drops and rebuilds all

	on a clean DB, live	3|7|5|11|6|0|0	7 tables, 36 constraints, functions, triggers, index, views and roles from scripts – re-runnable any time.
3–4	Start app + replay	Terminal 2: `uvicorn app.main:app --reload`. Terminal 1: `python -m app.replay data/replay_prices.csv --delay-ms 300 --run-id demo1`	Each price goes through ingest_tick(); watch the ALERT lines – 30 ticks, 8 alerts.
4–6	Web app tour + (4) trigger firing	Browser http://127.0.0.1:8000 → Dashboard → Alert History → Instruments (RELIANCE chart). Alert Rules: HDFCBANK ABOVE 1700, cooldown 0. Demo: HDFCBANK 1690 → Send, then 1710 → Send, then Resend.	1690: first tick, no alert. 1710 crosses 1700 → the AFTER INSERT trigger inserted the alert. Resend → DUPLICATE by the UNIQUE key.
6–7	(2) Role-based access	psql: `\i sql/screenshots/109_SEC5_least_privilege.sql`	SET ROLE sw_analyst can read prices, DELETE is denied; arjun only sees his alerts through v_my_alerts.
7–8	(3) Transaction with a deliberate failure	psql: `\i sql/screenshots/092_P2_sp_create_watchlist.sql` (and/or 027_X02_rollback.sql)	One unknown symbol → exception → nothing is created (0 'Banking' lists): all or nothing.
8–9	(5) Optimization before/after	Show IX2 and IX3 in the report (500,000-row benchmark DB).	Bitmap Heap Scan + Sort 19.966 ms → Index Scan 0.100 ms; column order explained.
9–11	(6) Concurrency	`python scripts/review3_concurrency_demo.py stock_watchlist`	B waits 2 s for A's lock; one alert, not two; duplicate event stored once.
11–12	Tests + wrap-up	Show R3_04 (or run `bash scripts/run_all_tests.sh` if time allows; it takes ~1 minute).	9 suites, 221 checks, all pass. Questions?

If something fails live: stay calm, say what you expected, and show the matching screenshot in the report (R3_01–R3_04, UI01–UI11). If the database is in a strange state, run the reset script and replay again (about 30 seconds).

Note: the concurrency demo script and the Review 3 helper scripts (`scripts/review3_*.py`, `scripts/review3_*.ps1`) were added for Review 3 – make sure they are copied into your project folder before the demo.

17. Walking through the report, section by section

Report section	What is in it	What to say / emphasise
Cover + Abstract	title, team, what the system does, what Review 3 adds	Problem → our solution → why PostgreSQL → Review 3 adds concurrency, testing, demo, UI screenshots.
1. Introduction	the problem and the database-driven approach	Most work happens inside the database; replay keeps the demo offline.
2. Project Scope	what is included / not included	Not a trading platform.
3. Key Contacts	team members	–
4. Resource Requirements	PostgreSQL 18, Python, FastAPI, Uvicorn, psycopg 3, HTML/CSS/JS, Git	Locked stack; no ORM.
5. ER Diagram	new Chen diagram, relationships, correction table	Section 4 of this guide: no arrows, CONTAINS is M:N not weak, no FK attributes.
6. Tables and Constraints	each table: description, columns, constraints, relationships, normal form, CREATE TABLE and \d screenshots; all constraints; normal forms with N01/N02	36 named constraints; CASCADE vs RESTRICT; every table BCNF with candidate keys.
7. SQL Implementation	DDL/DML (S01–S23, M01–M04), constraint rejections (CN01–CN06), transactions (X01, X02), 38 queries A–G, indexes + EXPLAIN (IX1–IX4)	Every screenshot is real psql output; B4 WHERE vs HAVING; D8 NOT IN pitfall; IX2 vs IX3.
8. Advanced SQL	CQ01–CQ10 with expected vs actual, views V1–V4, procedures P1–P3, functions U1–U2, triggers T1–T5, cursors CR1–CR4, security SEC1–SEC5, Review 2 checklist, changes since Review 1	Expected output was predicted before running; trigger edge cases (cooldown, late tick).
9. Review Requirements (Review 3)	checklist, 6-step demo flow, clean build (R3_01, R3_02), transactions & concurrency (R3_03), indexing & optimization, testing table + R3_04, working modules UI01–UI11, design changes since Review 2	This is the new part – sections 12–16 of this guide explain it.
10. Work break down	who did what	**Fill in before submitting.**
11. Review Evaluation	marks table (Project work with screenshot 5, Viva 5)	Left blank for the faculty.

Before submitting: fill in the 'Work Assigned' column (it still says [to be filled by the team]) and check the phone numbers in Key Contacts. Open the .docx in Word and right-click the table of contents → Update field if you change headings.

18. Viva questions with answers

These are the template's questions, answered for **our** project. Marking: 0 wrong, 1 definition only, 2 correct with example, 3 correct + justified + connected to our project – so always end with an example from our tables.

Design and normalization

Q. Why did you choose user_id as the primary key of users? What other candidate keys exist, and why were they rejected? Why not a natural key?

Candidate keys are user_id, username and email (all unique, all NOT NULL). We chose the surrogate user_id because it is short, never changes and is what the foreign keys point to. A username or email could change later; with a natural PK every referencing row (watchlists, alert_rules) would need updating. The natural keys are not lost – they are UNIQUE constraints uq_users_username and uq_users_email.

Same pattern elsewhere: instruments has PK instrument_id + UNIQUE (exchange, symbol); price_ticks has PK tick_id + UNIQUE (source, instrument_id, source_event_id).

Q. Give one functional dependency and prove it using attribute closure.

(exchange, symbol) → name. Closure of {exchange, symbol}: add instrument_id (by UNIQUE (exchange, symbol)), then name, quote_currency, is_active (instrument_id is the PK) → all columns. Since name is in the closure, the FD holds; since all columns are in it, (exchange, symbol) is a superkey (section 6.3).

Q. Show an update anomaly if two of your tables were merged. Which normal form prevents it?

Merge alert_events with price_ticks (store price and instrument in every alert). Correcting a tick's price would require changing every alert row that copies it – miss one and the data disagrees. tick_id → price, and tick_id is part of the key (rule_id, tick_id): a partial dependency, so **2NF** (and BCNF) prevents it. That is why alert_events has only event_id, rule_id, tick_id, fired_at.

Q. Is every 3NF relation in your schema also in BCNF? Give one relation where they could differ.

Yes, all 7 are BCNF: in every table the only determinants are candidate keys. 3NF and BCNF differ only when a column that is not a key determines part of a candidate key (this needs overlapping composite keys).

Hypothetical example: suppose each support analyst covers exactly one exchange, and we stored coverage(user_id, exchange, analyst) with the rules (user_id, exchange) → analyst and analyst → exchange. Candidate keys: (user_id, exchange) and (user_id, analyst). analyst → exchange: analyst is not a superkey, so it is **not BCNF**; but exchange is part of a key (prime), so it **is 3NF**. The fix is to split it into analyst_exchange(analyst, exchange) and user_analyst(user_id, analyst). None of our tables has this pattern.

Q. When would you intentionally denormalize? What risk do you take?

A latest_price table (one row per instrument) would make the dashboard read 7 rows instead of using the index. We chose not to: the index already makes 'latest price' fast (0.1 ms on 500,000 rows). The risk is the copy getting out of sync with price_ticks (update anomaly), and every tick insert would have to update two places. We would only do it if measurements showed the index was not enough.

Q. Your M:N relationship: what is the primary key of the junction table, and why?

watchlist_items has PK (watchlist_id, instrument_id). Each pair may appear only once – the same instrument cannot be in the same list twice (pk_watchlist_items rejects it, test A7). Both columns are also foreign keys. No surrogate id is needed because nothing references a single item.

Constraints and integrity

Q. For which FK did you choose CASCADE, and for which RESTRICT? What disaster if swapped?

CASCADE for user-owned data (users → watchlists, users → alert_rules, watchlists → watchlist_items, alert_rules → alert_events). RESTRICT for shared market data (instruments → price_ticks / watchlist_items / alert_rules, price_ticks → alert_events). Swapped: deleting one instrument would wipe its price history and everyone's alerts; deleting a user would be impossible until every list and rule was deleted by hand. See section 7.

Q. What happens when a parent row referenced by a child is deleted? Demonstrate.

Depends on the FK: DELETE FROM instruments WHERE symbol='RELIANCE' fails with 23001 (RESTRICT, screenshot CN04). DELETE FROM watchlists WHERE … succeeds and its watchlist_items rows disappear (CASCADE, CN05/M03) – but no instrument or tick is touched.

Q. Why does CHECK (qty > 0) not reject NULL, and how do you fix it?

CHECK rejects only FALSE. NULL > 0 is UNKNOWN, so it passes. Fix: add NOT NULL. Our price is NUMERIC NOT NULL CHECK (price > 0). volume is deliberately nullable (unknown volume) with CHECK (volume >= 0).

Q. DELETE vs TRUNCATE vs DROP – which can be rolled back, and do triggers / FK rules matter?

- DELETE removes chosen rows, fires row triggers, checks FKs (RESTRICT/CASCADE). Can be rolled back.
- TRUNCATE empties a whole table fast; no row triggers; it refuses if another table references it (unless CASCADE). In PostgreSQL it **can** be rolled back inside a transaction.
- DROP removes the table itself. PostgreSQL DDL is transactional, so DROP can also be rolled back – our benchmark drops the index inside a transaction and rolls it back (IX2).

In MySQL/Oracle TRUNCATE and DROP commit implicitly; in PostgreSQL they do not.

Queries

Q. Your query uses LEFT JOIN. Why not INNER JOIN? Show the row that would be lost.

C2 counts ticks for every instrument. HDFCBANK and M&M have no ticks after the replay. INNER JOIN would drop them; LEFT JOIN keeps them with a count of 0 (COUNT(t.tick_id) = 0).

Q. WHERE vs HAVING: rewrite one of your queries wrongly and explain.

B4: WHERE volume IS NOT NULL … GROUP BY instrument HAVING COUNT(*) >= 3. Putting COUNT(*) >= 3 in WHERE is an error ('aggregate functions are not allowed in WHERE') because WHERE runs before groups exist. Moving volume IS NOT NULL to HAVING is also wrong – volume is not grouped.

Q. NOT IN returns no rows because the subquery contains a NULL. Why, and the fix?

x NOT IN (a, b, NULL) means x<>a AND x<>b AND x<>NULL; x<>NULL is UNKNOWN, so the whole condition is never TRUE. D8: TCS volumes NOT IN (RELIANCE volumes) returns 0 rows because some RELIANCE volumes are NULL. Fix: NOT EXISTS, or add WHERE volume IS NOT NULL inside the subquery.

Q. EXISTS vs IN: when does the optimizer treat them differently? Which did you use?

Both become semi-joins in PostgreSQL in the simple case, so speed is usually the same. They differ for NOT: NOT IN has the NULL problem and can't be turned into an anti-join; NOT EXISTS can. We use EXISTS/NOT EXISTS for 'has this rule ever fired' (D6/D7) and IN for a short list (D3).

Q. Correlated subquery vs join: rewrite your correlated query as a join. Which is faster on 1M rows?

D5: ticks above their own instrument's average. Join version: compute AVG per instrument once in a derived table (GROUP BY instrument_id) and JOIN it. On 1M rows the join is usually faster because the

average is computed once per instrument instead of once per outer row (PostgreSQL does not always decorrelate it). Check with EXPLAIN ANALYZE.

Q. UNION vs UNION ALL: cost difference and when the wrong choice gives wrong results.

UNION removes duplicates (needs a sort or hash = extra cost); UNION ALL just appends. E2 counts how often each instrument is referenced by watchlists + rules – with UNION the duplicates would be removed and the counts would be wrong.

Q. COUNT(*), COUNT(col), COUNT(DISTINCT col) – predict the three results.

On price_ticks after the replay: COUNT(*) = 30 rows; COUNT(volume) = 22 (8 replay ticks have NULL volume); COUNT(DISTINCT instrument_id) = 5 (RELIANCE, TCS, INFY, BTCUSDT, ETHUSDT – HDFCBANK and M&M have none).

Indexes and optimization

Q. Why did you index (instrument_id, observed_at DESC, tick_id DESC) in that order? Will WHERE observed_at = ? use it?

Equality column first, then the ORDER BY columns, so 'latest N of one instrument' is one index range already in the right order. tick_id breaks ties. WHERE observed_at = ? alone cannot use it efficiently because observed_at is not the leading column (it would need a full index scan at best).

Q. Give a query where an index exists but will not be used. Why?

- On the demo data (30 ticks) PostgreSQL often uses a sequential scan – reading one small page is cheaper than the index.
- `WHERE price > 3000` – no index on price.
- `WHERE observed_at > now() - interval '1 hour'` without instrument_id – the leading column is missing.
- A function on the column, e.g. `WHERE date_trunc('day', observed_at) = ….`

Q. Explain your EXPLAIN output row by row. What join algorithm is used and why?

IX3: Limit → Index Scan using ix_price_ticks_instrument_time on price_ticks, Index Cond: instrument_id = 1, actual rows=50, no Sort node. IX2 (without the index): Limit → Sort (top-N heapsort) → Bitmap Heap Scan ← Bitmap Index Scan on uq_price_ticks_source_event. For joins, small lookups use Nested Loop with an index on the inner side; bigger unsorted inputs use Hash Join.

Q. Your table has 200 rows and the index gives no speed-up. Why? How would you prove the benefit?

With a few pages the whole table is read in microseconds, so the planner prefers a sequential scan. We proved the benefit on a copy with 500,000 ticks (tests/benchmark_indexes.py): 19.966 ms → 0.100 ms, buffers 6,371 → 16, medians of 7 runs.

Q. What is the cost of adding five more indexes to price_ticks?

Every tick INSERT must update every index (slower writes – and ticks are our most frequent write), more disk and memory (ours alone is 19 MB per 500,000 rows), more VACUUM work, and the planner has more choices to evaluate. Only add an index that a real, frequent query needs.

Transactions and concurrency

Q. Two customers buy the last item simultaneously (in our project: two feeds send prices for the same instrument at once). What happens at READ COMMITTED, and how do you prevent the problem?

At READ COMMITTED each session sees only committed data, so both could read the same 'previous price' and both create an alert (a duplicate). We prevent it with a row lock: ingest_tick() does SELECT … FOR NO KEY UPDATE on the instrument row, so the second session waits until the first commits and then sees its tick as the previous price. Demonstrated in R3_03: B waited 2.02 s, one alert only. (For 'last item in stock' the same idea: SELECT stock FOR UPDATE, check > 0, then UPDATE.)

Q. Explain a lost update with your own tables and two ways to prevent it.

Two admins open rule #1 (cooldown 300). Admin A sets 60, admin B sets is_active=false based on the old row and writes the whole row back – A's 60 is lost. Prevention:

- Locking: SELECT … FROM alert_rules WHERE rule_id=1 FOR UPDATE before changing it.
- Isolation: REPEATABLE READ – the second writer gets a serialization error and retries.
- Optimistic versioning: a version column; UPDATE … WHERE rule_id=1 AND version=7, and retry if 0 rows.

Our API avoids it by updating only the changed column (PATCH sets just is_active or cooldown_seconds).

Q. Dirty, non-repeatable and phantom reads – which isolation level prevents each? Which remain at your default level?

See the table in section 12.3. At our default READ COMMITTED, dirty reads cannot happen; non-repeatable and phantom reads can. PostgreSQL never allows dirty reads even at READ UNCOMMITTED.

Q. How does PostgreSQL guarantee durability after a power failure?

Write-ahead logging: every change is written to the WAL first, and at COMMIT the WAL is flushed to disk (fsync). Data pages are written later; checkpoints mark a point up to which all pages are on disk. After a crash PostgreSQL replays the WAL from the last checkpoint, so every committed transaction is restored and uncommitted ones are discarded.

Q. How could a deadlock occur in ingest_tick (or a similar procedure)? How would you avoid it?

ingest_tick() locks only one instrument per call, so it cannot deadlock with itself. A deadlock would need a transaction that ingests RELIANCE then TCS while another ingests TCS then RELIANCE – each waits for the other. PostgreSQL detects it after deadlock_timeout and aborts one (error 40P01). Avoid it by always locking in the same order (e.g. by instrument_id) and keeping transactions short; retry on 40P01.

Procedural SQL and security

Q. Why is your business rule in a stored procedure/function and not in the application? When is a trigger worse? Give a trigger hazard.

ingest_tick() is shared by the replay, live feed and API – one implementation, the same lock and the same duplicate rule for everyone, and it runs next to the data in one round-trip.

A trigger is worse when the logic is surprising or expensive: it runs on every insert, it is hidden from someone reading the INSERT, and a trigger that writes to a table with its own trigger can recurse. Our alert trigger is acceptable because firing alerts on every tick IS the rule, and it only inserts into alert_events (whose trigger only checks, never inserts).

Q. Function vs procedure: why is fn_latest_price a function? Can it be used in SELECT? Can it contain a transaction?

It returns one value and we want to use it inside queries (CQ02 uses it per watchlist item), so it is a function – yes, usable in SELECT. A function cannot COMMIT/ROLLBACK; it runs inside the caller's transaction. A procedure (CALL) can control transactions.

Q. Why would a set-based UPDATE beat your cursor? When is a cursor still justified?

One UPDATE … WHERE … lets PostgreSQL process all rows in one pass; a cursor loop sends one statement per row (much slower). A cursor is justified when you must process rows one by one with per-row logic or output (our fn_rule_status_report builds a line per rule), or to stream a huge result without loading it all (CR4 SCROLL cursor).

Q. A login form builds SELECT * FROM users WHERE name='"+input+"'. Craft the injection and two defenses. Why doesn't GRANT alone protect?

Input: `' OR '1'='1` → WHERE name='' OR '1'='1' returns every user. Defenses:

- Parameterized queries: our whole backend uses psycopg with %s placeholders, never string building (API test_29 stores an injection string as plain text).
- Least privilege + input validation: the app role can only touch what it needs; Pydantic validates types and lengths.

GRANT does not help because the injected SQL runs with the application's own rights – which are allowed to read users.

Q. A staff member should see only their branch's data. Design it with views/roles/privileges.

Exactly our v_my_alerts pattern: a view WITH (security_barrier) that filters WHERE owner = current_user, GRANT SELECT on the view only, no rights on the base table. For branches: a staff table mapping role → branch, and the view joins it on current_user. (Alternative: Row-Level Security policies.)

Q. How are passwords stored? What is wrong with MD5(password)?

Our app has no login (a demo user selector), so it stores no passwords. PostgreSQL itself stores role passwords as SCRAM-SHA-256 (salted, many iterations) – this laptop's server uses scram-sha-256. MD5 is fast and unsalted, so identical passwords give identical hashes and attackers can try billions per second or use precomputed tables. Use a slow salted hash (bcrypt, scrypt, Argon2, SCRAM).

Q. A table is accidentally emptied at 3:15 PM. How do you recover to 3:14 PM?

Point-in-time recovery: restore the last base backup (pg_basebackup) and replay the archived WAL with recovery_target_time = '… 15:14:00'. That needs WAL archiving to be switched on beforehand. With only pg_dump you can restore the last dump, losing changes since then. For our demo database the reset script rebuilds everything from the SQL files in seconds.

Questions about our project they may add

Q. Why PostgreSQL?

Enforced CHECK constraints (with regex), exact NUMERIC, TIMESTAMPTZ, PL/pgSQL triggers that run in the same transaction, row locks, pg_notify, transactional DDL, roles and security_barrier views – all used in this project.

Q. Why is the first tick never an alert?

A crossing needs a previous price. With no previous price we cannot know which side it came from.

Q. Why cooldown in market time and not wall-clock time?

Replays and late feeds would otherwise fire on the wrong spacing. Test 10d shows three alerts share one fired_at, so fired_at could not drive the cooldown; observed_at does.

Q. What exactly is alert_events.fired_at? Does tick_id → fired_at break 2NF?

fired_at is **the time the alert row was recorded** (DEFAULT now(), the start of the recording transaction). Today every alert is written inside the tick's own transaction, so fired_at equals the tick's ingested_at (8

of 8 in the replay) – but that is how the current code writes, not a rule about the data. An event recorded later or corrected by hand could have a different fired_at, and the schema allows it. So tick_id → fired_at is **not** an FD of the design, and alert_events stays in BCNF.

Do not say "fired_at is the tick's time" or "alert_events violates 2NF". (Team decision in Phase 13; evidence docs/review2/sql_output/21_fired_at_analysis.out.)

Q. Did the database schema change between Review 1 and Review 2?

No table, constraint, function, trigger or index changed in the Review 2 work (Phase 13 compared checksums of every existing file before and after). What changed was the **documentation**: the Review 1 ER diagram and tables section were wrong or incomplete, and were corrected to match the database (section 20).

Q. What is pg_notify for?

After an alert is inserted the trigger sends only the event_id on channel alert_events, so a listener could push it to the browser. The UI currently uses 5-second polling ('Live refresh').

19. Honest limitations – what not to claim

- No real login: the user selector is a demo choice, not authentication. Security is shown with database roles.
- SET ROLE is not a login – say 'we switch to the role's privileges'.
- The same-instrument rule on alert_events is checked when alerts are written. If someone later changed instrument_id on a rule or tick by hand, the old alerts would not be re-checked. No application path does that (the API can only change is_active and cooldown). A fix (a trigger that refuses an instrument_id change on alert_rules / price_ticks) was written and tested in a throwaway database in Phase 13 but is **not applied** – say 'proposed', not 'done' (docs/review2/PROPOSED_INTEGRITY_FIX.md).
- The index shows no visible gain on 30 demo ticks – the benefit is measured on 500,000 rows.
- Do not claim the system 'handles hundreds of instruments per second' – nothing measured that.
- Replay ticks are stamped into the future; that is why the live feed refuses to start after a replay.
- Tests on Windows need the two small test-file adjustments described in section 14.

20. What changed in Phases 11, 12 and 13

Phases 1–10 built the system. Phases 11–13 (all on 21 Oct 2026) are clean-up and Review 2 corrections. **None of them changed a table, constraint, function, trigger, index or API behaviour.** If asked "what did you change after Review 2?", the answer is the documentation and the evidence, plus the Review 3 work in sections 12–16.

Phase	What changed	What it means for the viva
11	docs/PHASE1_SPEC.md renamed to **docs/SCHEMA_DESIGN.md** (the design document: every column, key and constraint). Old internal working notes deleted: FACT_SHEET, VIVA_NOTES, DEMO_GUIDE, EVIDENCE_CHECKLIST, RESEARCH_BRIEF, RUBRIC and the agent CLAUDE.md.	If you point to the design, point to docs/SCHEMA_DESIGN.md. The viva answers now live in this guide and docs/review2/VIVA_RECOVERY.md.
12	'Phase N' labels removed from code comments, doc titles, test names and the test runner (it now prints 'Schema tests', 'Alert tests', … instead of 'Phase 2 …').	Cosmetic only – same tests, same counts (51, 50, 17, 18, 15, 32, 14, 24).

	Throwaway test databases renamed (e.g. stock_watchlist_replay_test, stock_watchlist_livefeed_test). verify_spec.sql now prints "LIVE SCHEMA MATCHES docs/SCHEMA_DESIGN.md". inspect_schema.sql: one ORDER BY fix and a heading that now lists the extra index.	
13	Review 2 corrections and evidence: corrected Chen ER diagram, relational schema diagram, normalization proof, constraint inventory read from the catalog, change log, 22 SQL demonstration files (sql/review2/) with their real outputs (docs/review2/sql_output/), and helper scripts (scripts/review2_*). The demonstrations run only in the throwaway database stock_watchlist_review2_sandbox, in schema review2.	This is where the ER fixes, the 2NF/3NF wording and the fired_at decision come from (below).

20.1 What the Review 1 documents got wrong (fixed in Phase 13)

The database was already right; the Review 1 ER diagram and tables section did not match it.

Problem in Review 1	Correction
Arrow heads on attribute and relationship lines	Plain lines; cardinality as 1 / N / M
No total participation shown	Double lines where the FK is NOT NULL (all 8 FKs are)
Watchlist_item drawn as a weak entity	M:N relationship CONTAINS with attribute added_at
ADDED_TO linked Instruments to Watchlist_item	One relationship CONTAINS between WATCHLIST and INSTRUMENT
Only surrogate ids underlined	username, email and listing (exchange, symbol) also shown as keys
Mixed entity names (User / Watchlists / Price_Ticks …)	Singular, one style: USER, WATCHLIST, PRICE_TICK …
Spelling quote_currentcy	quote_currency
ER and tables section disagreed in 8 places (e.g. enabled vs is_active, alert_rule_id vs rule_id)	Both follow the real catalog; the database was not renamed
Missing UNIQUE, CHECK and ON DELETE rules in the tables section	All 36 named constraints listed (7 PK, 7 UNIQUE, 8 FK, 14 CHECK)
volume NOT NULL; price as plain NUMERIC	volume is NULL-able (unknown ≠ 0); numeric(18,8) / numeric(24,8)
No normalization section	Candidate keys, FDs and 1NF–BCNF for every table

docs/ER_DIAGRAM.md is now the **relational schema** diagram (Crow's Foot: tables, PK/FK). The conceptual ER diagram is the Chen one (report Figure 5.1). Composite alternate keys there are marked AK1, e.g. AK1 (exchange, symbol) – the columns are unique **together**, not each on its own.

20.2 Normalization statements that were corrected

- "exchange → asset_class would break 3NF" → it would break **2NF** (exchange is part of the key (exchange, symbol), so it is a partial dependency). It is also not true in general (NSE lists ETFs).
- "Storing price in alert_events breaks 3NF" → it breaks **2NF** (tick_id is part of the key (rule_id, tick_id)); seen from the PK alone it is also transitive.

- "BSE RELIANCE disproves symbol → name" → invalid (same company, same name). symbol → name is simply not adopted as a rule.
- "tick_id ↛ fired_at because one tick fires several rules" → wrong reason (those events share one fired_at). The real reason: fired_at is the event row's recording time.
- Rolled-back inserts do not *prove* an FD false; they show the database does not enforce it. FDs are accepted or rejected from the business rules.

20.3 Decisions recorded in Phase 13

- **fired_at kept unchanged** and defined as the alert row's recording time (section 18, last questions).
- **Parent-update gap of the same-instrument rule:** fix proposed and tested, **not applied** (section 19).

21. Where everything is (file map)

Path	What
sql/00_schema.sql	7 tables + 36 constraints (drops and recreates)
sql/01_seed.sql	demo data
sql/03_functions_triggers.sql	ingest_tick() + 2 triggers
sql/06_indexes.sql	ix_price_ticks_instrument_time
sql/09_review2_objects.sql	schema reporting: views, procedures, functions, cursors, roles
sql/screenshots/*.sql	one SQL file per report screenshot
scripts/reset_demo_db.sh	rebuild the database to the clean seed (3|7|5|11|6|0|0)
scripts/run_all_tests.sh	run all 9 test suites
scripts/review3_concurrency_demo.py	the two-session concurrency demo (R3_03)
scripts/review3_er_diagram.py	redraws docs/review3/ER_DIAGRAM_REVIEW3.png
scripts/review3_ui_screenshots.py	re-takes UI01–UI11 with Playwright (needs a throwaway DB)
scripts/review3_capture.ps1	captures one console window as PNG (R3_01–R3_04)
scripts/review3_build_report.py / review3_word_update.ps1	rebuild Review 3 Report.docx and its PDF
docs/review3/screenshots/	R3_01–R3_04 and UI01–UI11
docs/screenshots/	Review 2 SQL screenshots (S, M, CN, N, A–G, CQ, V, P, U, T, CR, SEC, IX)
docs/SCHEMA_DESIGN.md	the schema design: every column, key and constraint (was PHASE1_SPEC.md)
docs/review2/	Review 2 corrections: change log, normalization proof, constraint inventory, viva notes
sql/review2/ + docs/review2/sql_output/	Review 2 SQL demonstrations and their real outputs (sandbox DB only)
app/	FastAPI backend (main.py), replay.py, live_feed.py, static/ web page
data/replay_prices.csv	the 30 replay prices